

QR Payment Platform

Current-State System Design

Cloud & Infrastructure Architecture Review

Prepared for: Head of Infrastructure · For architecture review

Project Details

Project Name	: <i>QR Payment Platform — Banking App and Merchant (E-commerce) App</i>
Document Type	: <i>Current-State (As-Is) System Design · Infrastructure & Cloud Architecture</i>
Prepared By	: <i>Cloud Architecture Study Team</i>
Reviewed With	: <i>Head of Infrastructure</i>
Repository	: <i>github.com/jmbilbao25/Banking-App</i>
Environments in Scope	: <i>AWS EC2 (Dev/Test) · Azure Container Instances (Prod) · Jenkins CI/CD</i>
Scope and Objectives	: <i>Establish a verified, evidence-based baseline of the platform as it runs today, so that platform investment and a cloud target state can be argued from fact rather than from the README.</i> • <i>Document the deployed runtime topology across both cloud environments</i> • <i>Trace the end-to-end payment flow hop by hop, including ports, protocols and timeouts</i> • <i>Record the delivery pipeline, network posture, data tier and observability position</i> • <i>Assess resilience, scalability and recoverability against a production operating bar</i> • <i>Identify the architecture-level gaps that must close before any production traffic</i>

All statements in this deck were reconstructed from the repository contents and from the prototype executed end to end in a sandbox, not from documentation.

Design Considerations

- Design Approach** : *Two independently deployable Python/Flask services — a banking service and a merchant service — integrated over synchronous REST. Each owns its own relational database. Containers are built once and promoted from AWS Dev to Azure Prod by a Jenkins pipeline.*
- Design Alternatives** : *Kubernetes manifests for both services already exist in the repository but are not the deployed path; production currently runs on Azure Container Instances. AKS is therefore an available, partially prepared alternative rather than a greenfield decision.*
- Constraints and Limitations** :
 - *Prototype was built to demonstrate a payment journey, not to meet an operating standard*
 - *Azure Container Instances offers no ingress, autoscaling or rolling-update primitive*
 - *No infrastructure-as-code — cloud resources are created by imperative CLI calls in CI*
 - *Single-node databases in both environments; no managed HA tier is provisioned*
- Dependencies** : *Docker Hub (image distribution) · Jenkins (build and deploy orchestration) · Azure subscription (ACI, Azure Database for MySQL) · AWS account (EC2 Dev/Test host).*
- Assessment Basis** : *Repository inspection, container and pipeline definitions, plus a live end-to-end execution of both services to confirm the runtime behaviour documented here.*

Design Considerations

Assumptions

Assumption	Identified by	Impact if assumption is proven invalid
The Azure and AWS environments in the pipeline are non-production / lab subscriptions.	Study Team	If any is a production subscription, the public endpoints and inline pipeline credentials become an immediate operational exposure and must be remediated before this review proceeds.
Existing container images can be rebuilt unchanged onto a managed orchestrator (AKS or Container Apps).	Study Team	Additional refactoring effort is required — chiefly externalising session state and the service-to-service addressing that is currently passed as environment variables.
The published Kubernetes manifests represent the team's intended target runtime.	Study Team	The migration path must be re-planned; effort estimates based on reusing those manifests would not hold.
Reported transaction volume is prototype-scale, with no committed throughput or latency target.	Study Team	Capacity, autoscaling and database sizing must be re-derived from a real volume forecast before any sizing in this deck is used.
The two services are intended to remain separately deployable, not merged into one application.	Study Team	The integration analysis and the network segmentation design would both be restated around a single deployable unit.

Design Considerations

Architecture and Platform Risks

#	Risk Description	Exposure	Mitigation
R-1	Every tier is a single instance behind a single DNS name. There is no load balancer distributing traffic across the two production containers.	Availability	Introduce a managed ingress (Application Gateway / Front Door) and place both replicas behind a health-probed backend pool.
R-2	Deployment recreates each container (delete then create), so every release is a full outage of that service.	Availability	Move to a platform with rolling updates and readiness gating — AKS (manifests already exist) or Azure Container Apps.
R-3	Single-node databases with no replica, no automated backup and no point-in-time recovery. RPO and RTO are undefined.	Data loss	Managed database with zone-redundant HA, automated backups and a tested restore. Define and sign off RPO/RTO.
R-4	All traffic, including service-to-service calls, is cleartext HTTP on port 80. No TLS anywhere in the path.	Confidentiality	TLS termination at a managed edge, HTTPS end to end, and private networking for east-west traffic.
R-5	Two databases are committed in sequence with no distributed transaction, queue, retry or reconciliation. A timeout leaves them permanently disagreeing.	Integrity	Asynchronous integration over a managed broker with an outbox, idempotent consumers and a scheduled reconciliation job.
R-6	Images are published to a public registry under a mutable :latest tag, unsigned and unscanned. What is running cannot be proven.	Supply chain	Private registry (ACR) with immutable digests, vulnerability scanning and signed images gated in the pipeline.
R-7	No infrastructure-as-code; environments are produced by imperative CLI calls, so they cannot be reproduced or audited.	Operability	Declarative IaC (Bicep or Terraform) under version control, applied by pipeline with drift detection.
R-8	No metrics, tracing, structured logging or alerting. An incident cannot be detected proactively or reconstructed afterwards.	Operability	OpenTelemetry to Azure Monitor / App Insights, correlation IDs across services, dashboards and SLO-based alerts.

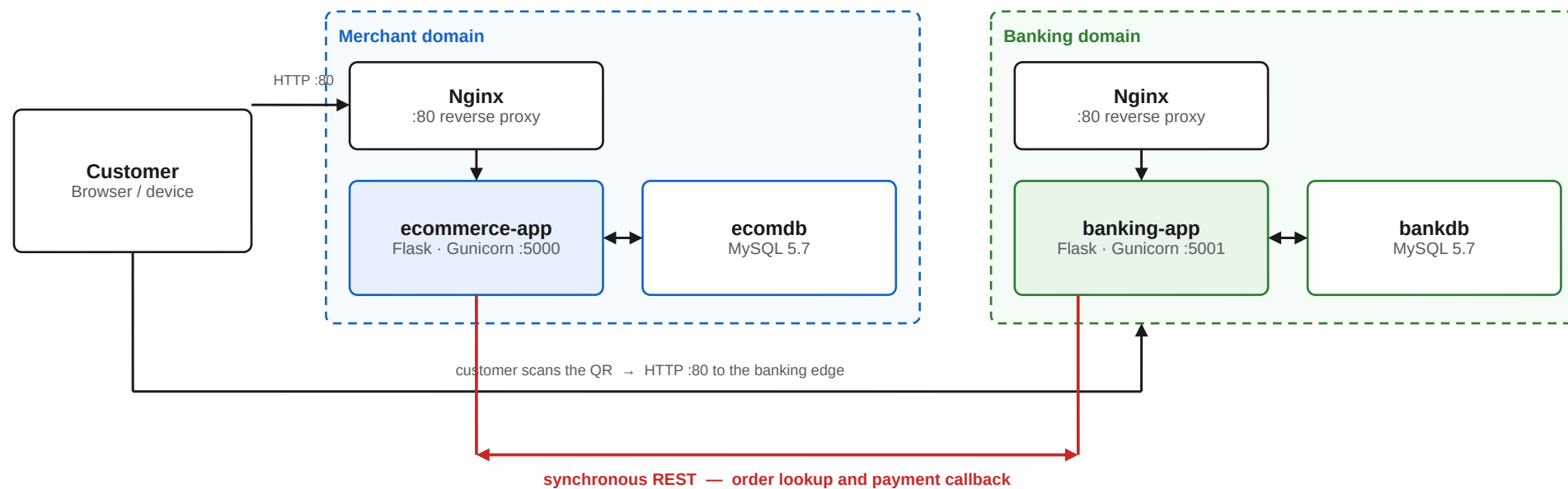


Solution Overview

System context and component responsibilities

Description/s

- : A customer shops on the merchant service, which issues a QR code encoding a payment URL addressed to the banking service. The customer authorises the payment in the banking service, which then calls the merchant service back to release the order.



Integration shape

Bidirectional, synchronous, cleartext HTTP.

Addressing is by environment variable, not service discovery.

No broker, no retry, no circuit breaker.

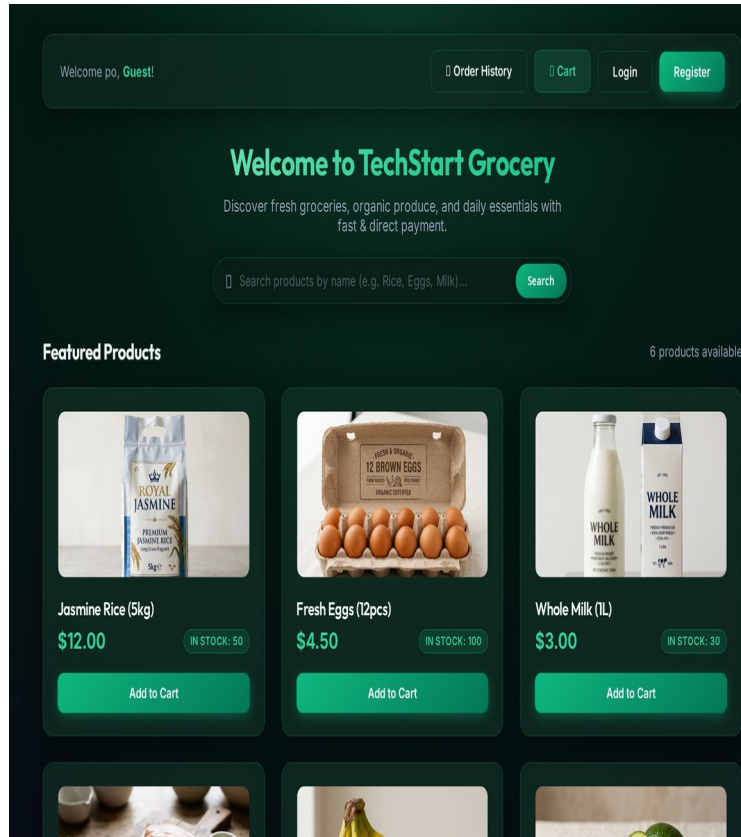
In production this path leaves the trust boundary.

Both services also expose /health, and container probes are defined for it in the unused Kubernetes manifests.

User Interfaces

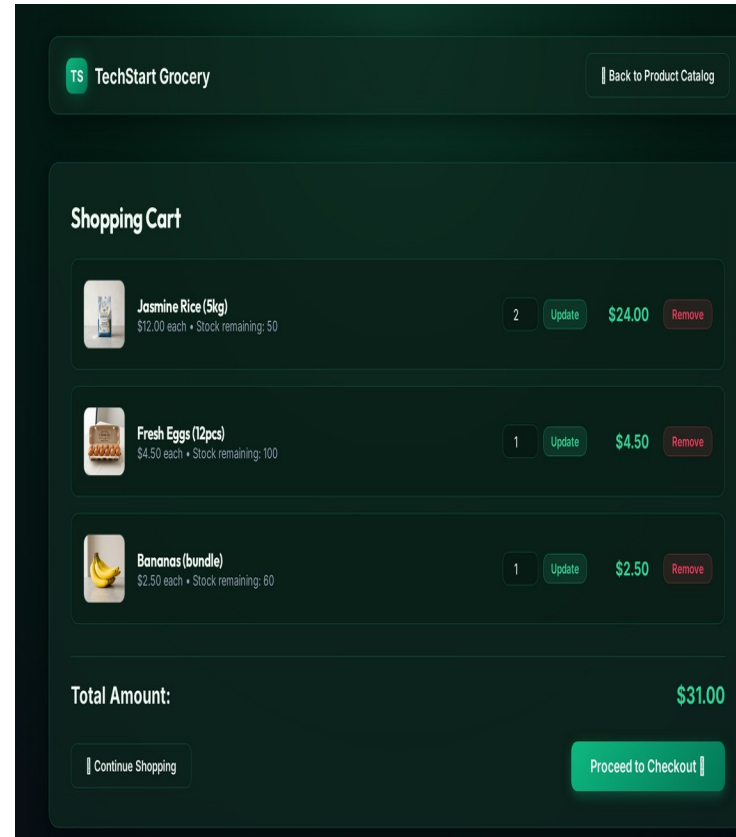
Merchant service — catalogue to QR presentment

Product catalogue



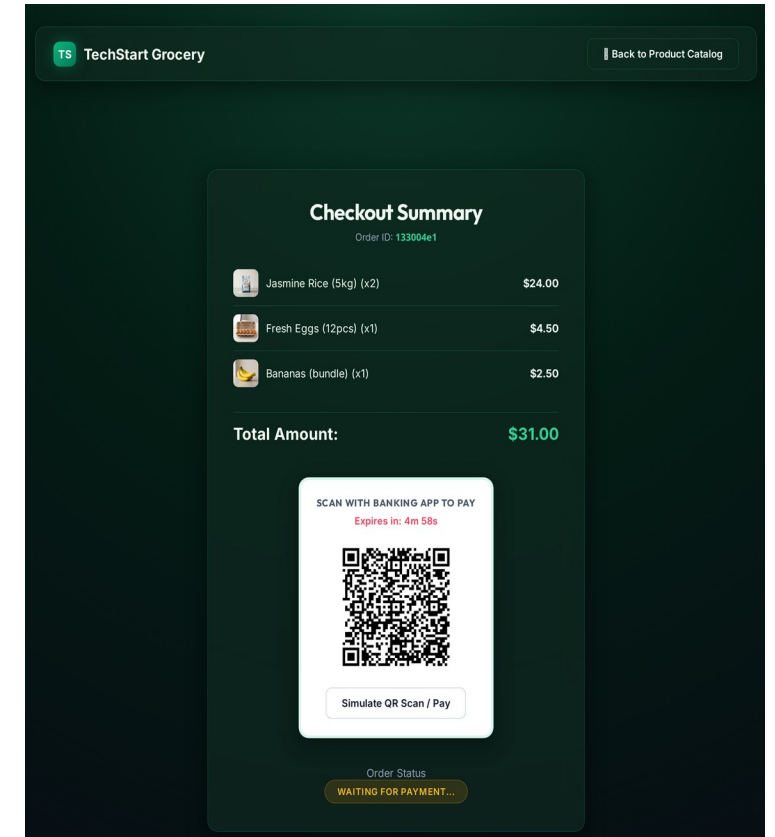
Basket state is held in the signed session cookie — no server-side session store.

Basket review



Stock is re-validated against the database on every basket mutation.

Checkout and QR presentment



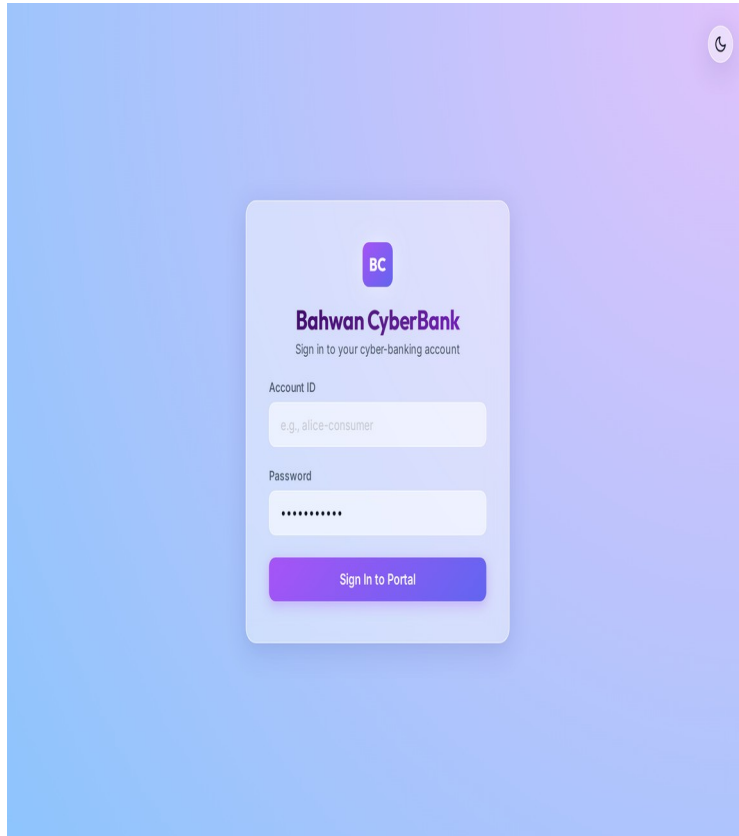
Order is created PENDING with a 300-second expiry; the page polls status every 2 s.

Screens captured from the prototype running end to end; order 133004e1, total 31.00.

User Interfaces

Banking service — authorisation to settlement

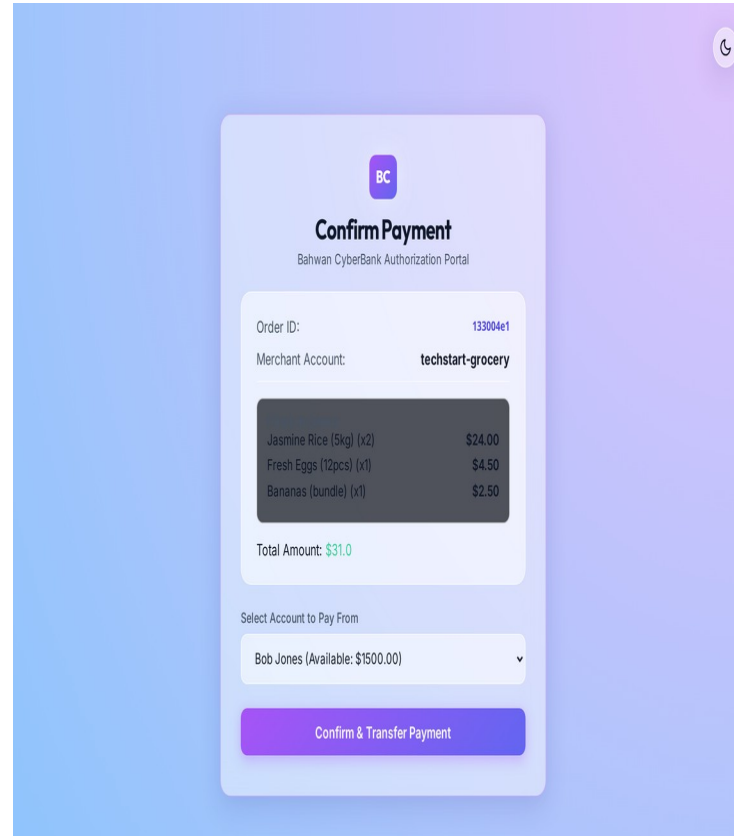
Authentication



The login screen features the Bahwan CyberBank logo and a sign-in form. The form includes fields for Account ID (with the example 'e.g., alice-consumer') and Password (masked with dots). A 'Sign In to Portal' button is at the bottom.

Scanning the QR without a session stores next_url and redirects to login.

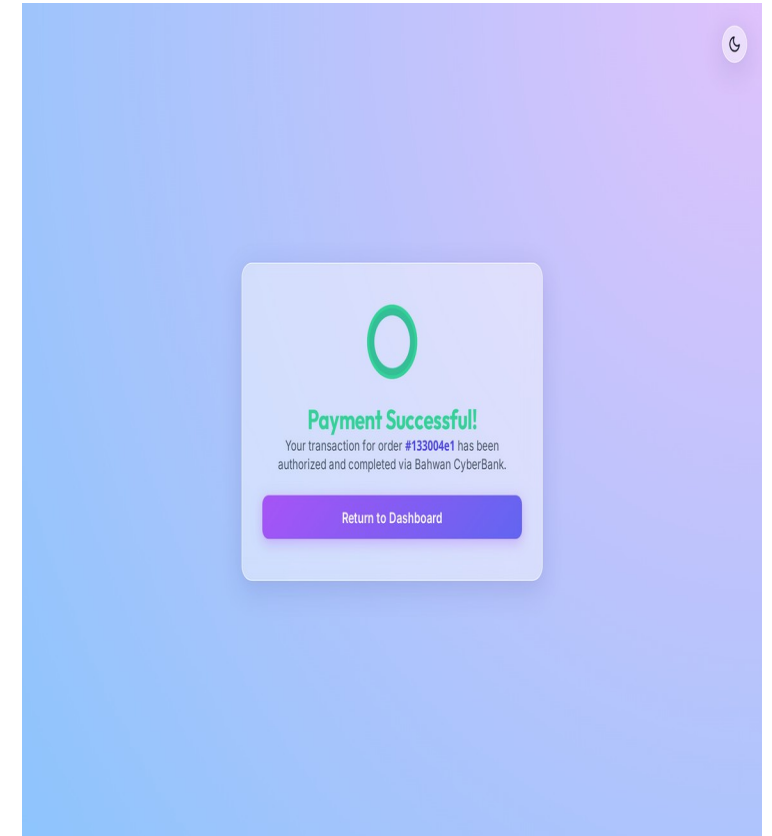
Payment authorisation



The 'Confirm Payment' screen displays transaction details. It includes the Order ID (133004e1) and Merchant Account (techstart-grocery). A table lists the items: Jasmine Rice (5kg) (x2) for \$24.00, Fresh Eggs (12pcs) (x1) for \$4.50, and Bananas (bundle) (x1) for \$2.50. The total amount is \$31.0. Below the table, a dropdown menu shows 'Bob Jones (Available: \$1500.00)' as the selected account to pay from. A 'Confirm & Transfer Payment' button is at the bottom.

Order lines are fetched live from the merchant service with a 2-second timeout.

Settlement confirmation



The 'Payment Successful' screen shows a large green checkmark icon. Below it, the text states: 'Payment Successful! Your transaction for order #133004e1 has been authorized and completed via Bahwan CyberBank.' A 'Return to Dashboard' button is at the bottom.

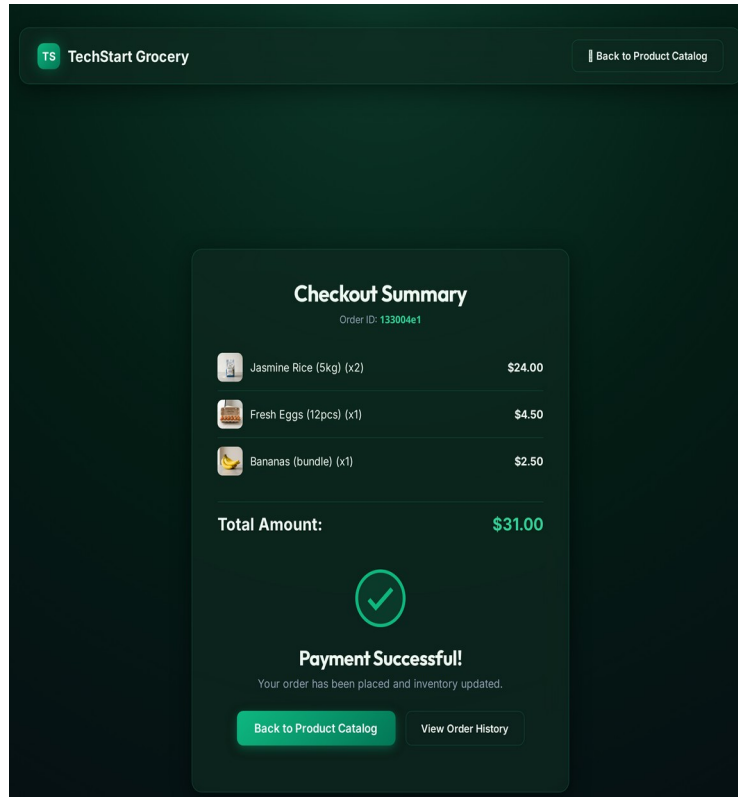
Rendered only after the merchant callback and the ledger commit have both returned.

Session cookies are signed with an application-local key, so they are not portable between replicas.

User Interfaces

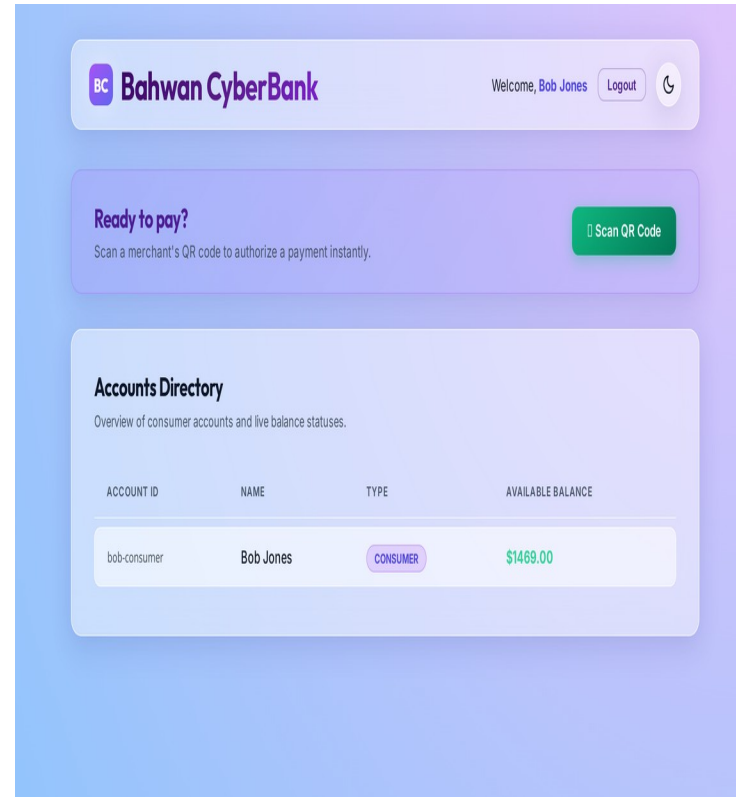
Post-settlement state across both services

Merchant order converges to PAID



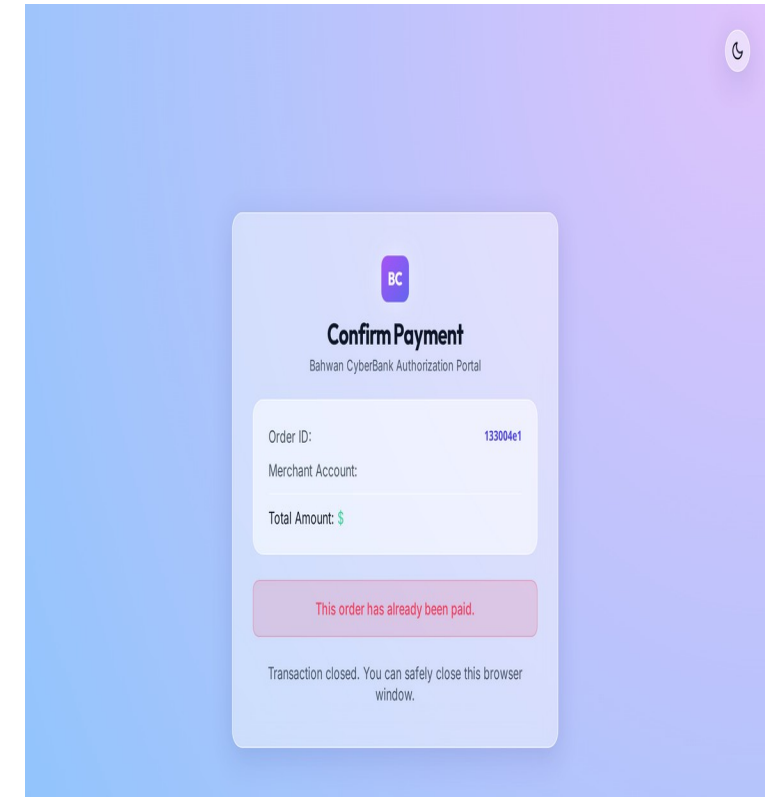
Reached only by the 2-second client poll — the merchant is never pushed a notification.

Ledger balance after debit



Payer balance moved 1,500.00 to 1,469.00 for the 31.00 order.

Replay of the same QR

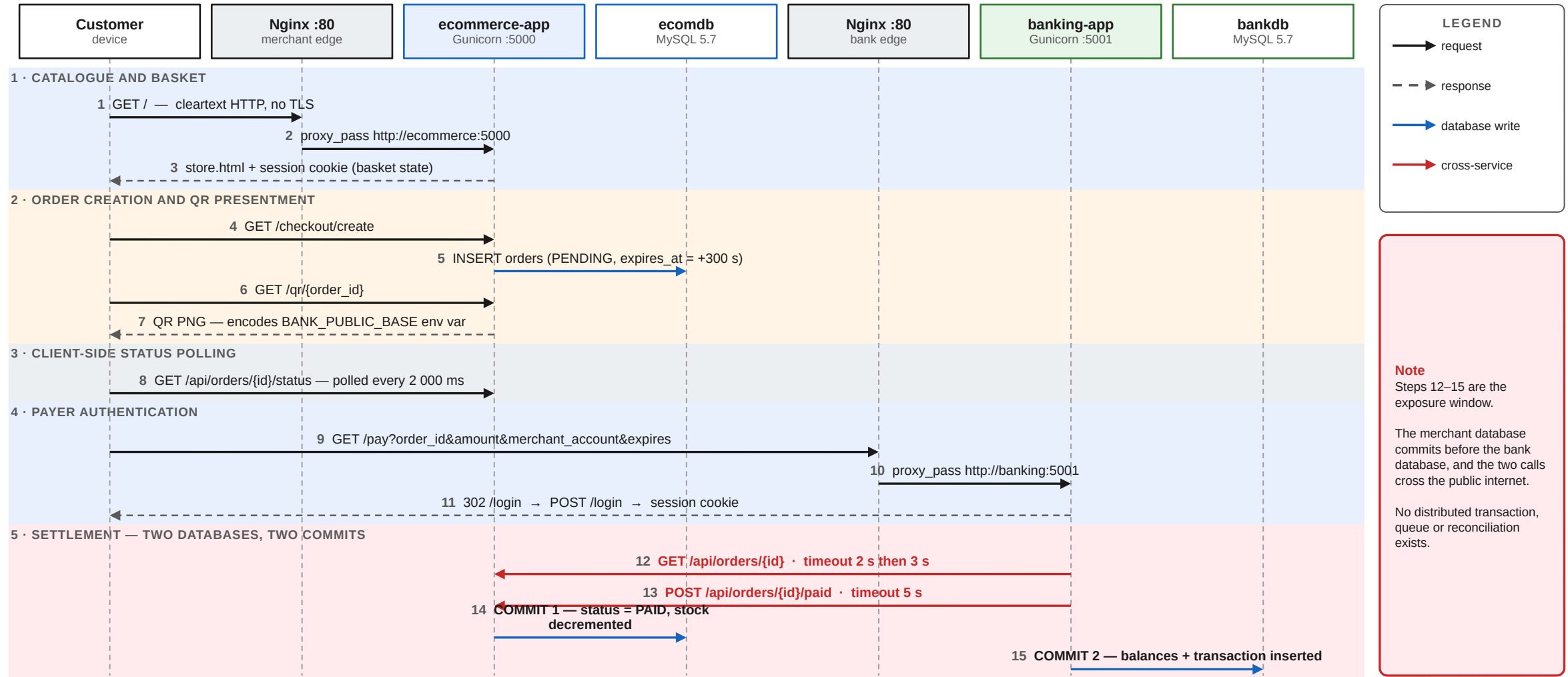


A second presentment of the same order is refused by the duplicate check.

State convergence between the two services depends entirely on client-side polling; there is no server-to-server notification or event.

Runtime Sequence

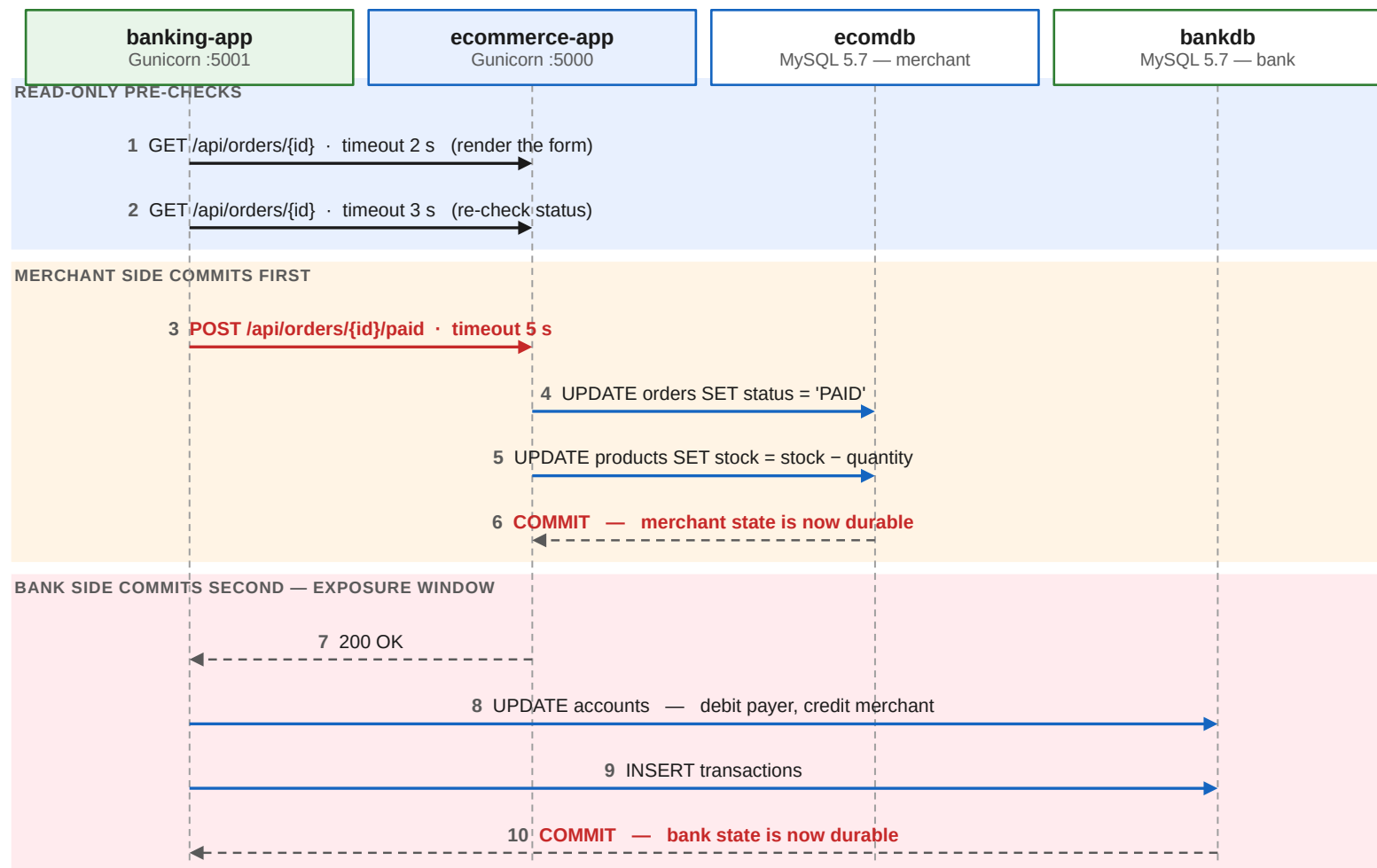
End-to-end QR payment — every network hop as deployed today



Reconstructed from banking-app/app.py, ecommerce-app/app.py and nginx/nginx.conf; confirmed against a live run.

Runtime Sequence

Settlement call chain — commit ordering and the exposure window



Timeout values are the literal values configured in the banking service's HTTP client calls.

What the ordering guarantees

The merchant's order is marked PAID and stock is decremented BEFORE any money moves. Once step 6 commits, that outcome is durable and irreversible.

What fails in the window

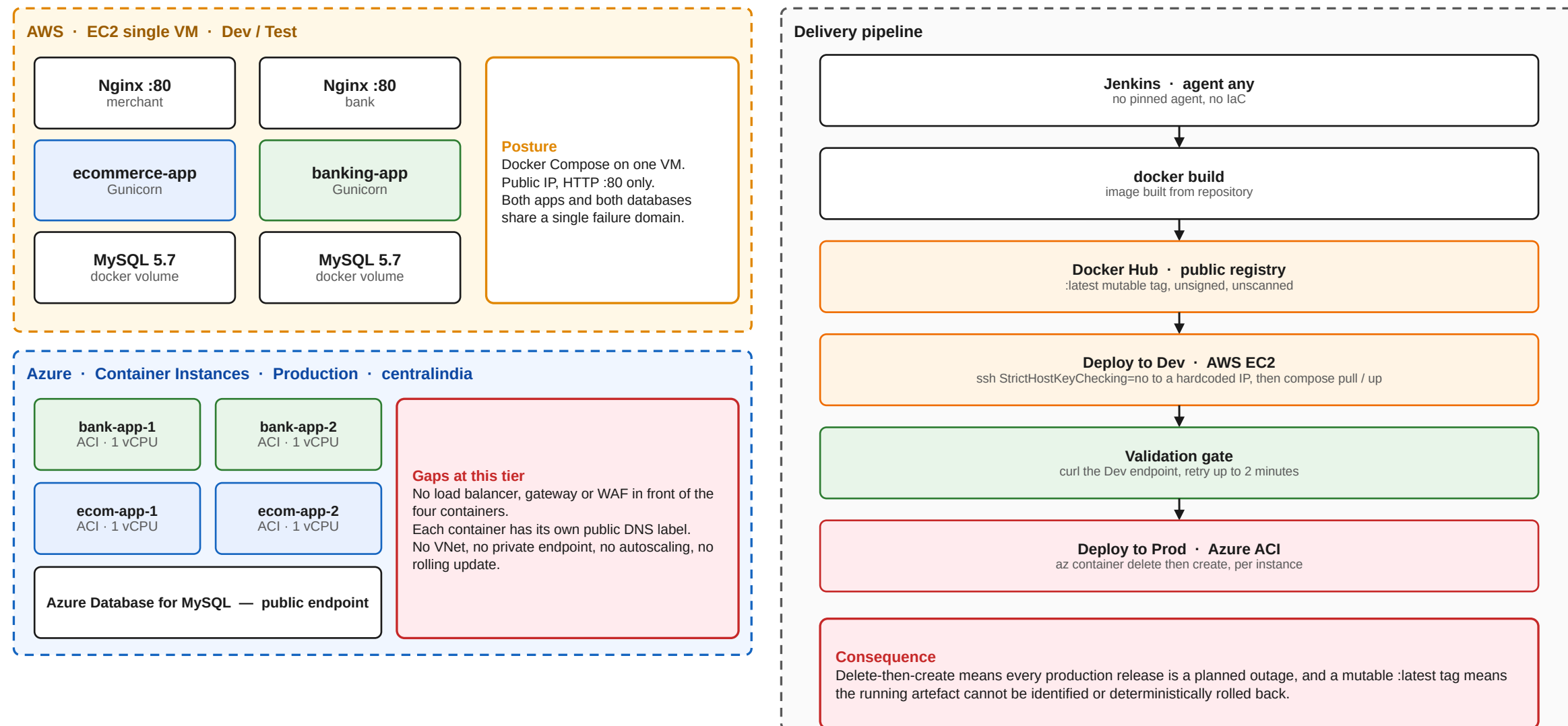
Any fault between steps 6 and 10 — database failover, container eviction, a lost response, a 5 s timeout that in fact succeeded — leaves the order PAID with no corresponding ledger entry.

Platform capability required

Transactional outbox, a managed broker with at-least-once delivery, idempotent consumers, and a scheduled reconciliation sweep.

Infrastructure and Technology Stack

Current deployment topology across both cloud environments



System Interfaces

Interface inventory and integration characteristics

Description/s : *All integration is synchronous REST over cleartext HTTP. There is no message broker, no API gateway and no service mesh in the current design.*

Source	Destination	Interface	Type	Frequency	Transport / posture
Customer device	Nginx → ecommerce-app	GET / · /cart · /checkout/*	Synchronous HTML	Interactive	HTTP :80, cleartext
Customer device	ecommerce-app	GET /api/orders/{id}/status	Synchronous JSON	Every 2 000 ms per open checkout	HTTP :80, cleartext
Customer device	ecommerce-app	GET /qr/{order_id}	Image (PNG)	Once per order	HTTP :80, cleartext
Customer device	Nginx → banking-app	GET/POST /pay · /login	Synchronous HTML	Interactive	HTTP :80, cleartext
banking-app	ecommerce-app	GET /api/orders/{id}	Synchronous JSON	Twice per payment	HTTP, public internet, 2 s / 3 s timeout
banking-app	ecommerce-app	POST /api/orders/{id}/paid	Synchronous JSON	Once per payment	HTTP, public internet, 5 s timeout
Container platform	Both services	GET /health	Liveness / readiness	Defined, never executed	Only in the unused k8s manifests
Each service	Its own database	SQLAlchemy / PyMySQL	Database session	Per request	In-cluster or public MySQL endpoint

Interface observation

The two highlighted rows are the only east-west calls in the system, and they carry the settlement decision. In production they traverse the public internet with no private link, no mutual authentication and no retry.

Infrastructure and Technology Stack

Component versions and support position

Description/s : *Python/Flask services packaged as single-stage Docker images, fronted by Nginx and persisted to MySQL. Versions below are the versions pinned in the repository.*

Layer	Technology	Version	Role	Support position
Runtime	Python	3.11 (slim base image)	Application runtime	Supported
Framework	Flask	3.0.3	Web framework for both services	Supported
Framework	Flask-SQLAlchemy	3.1.1	ORM and data access	Supported
App server	Gunicorn	21.2.0	WSGI server, bound to container port 80	Supported
Edge	Nginx	Container image, :80	Reverse proxy; no TLS configured	No TLS termination
Database	MySQL	5.7	Relational store, one per service	Reached end of life October 2023
Database	Azure Database for MySQL	Managed, public endpoint	Production data tier	Supported; not privately networked
Fallback store	SQLite	Bundled with Python	Silent fallback when DB_HOST does not resolve	Not fit for production use
Container	Docker	Single-stage build, runs as root	Image packaging	USER directive present but commented out
Orchestration (Dev)	Docker Compose	v3.8 file format	Single-VM orchestration on EC2	No HA, no scheduler
Orchestration (Prod)	Azure Container Instances	Managed	Runs four containers, two per service	No ingress, autoscale or rolling update
Orchestration (defined)	Kubernetes manifests	apps/v1	Deployment, Service, Secret — present in repo	Not deployed; Service is type LoadBalancer
CI/CD	Jenkins	Declarative pipeline, agent any	Build, promote and deploy	Supported
Registry	Docker Hub	Public repository	Image distribution	Mutable :latest, unsigned, unscanned

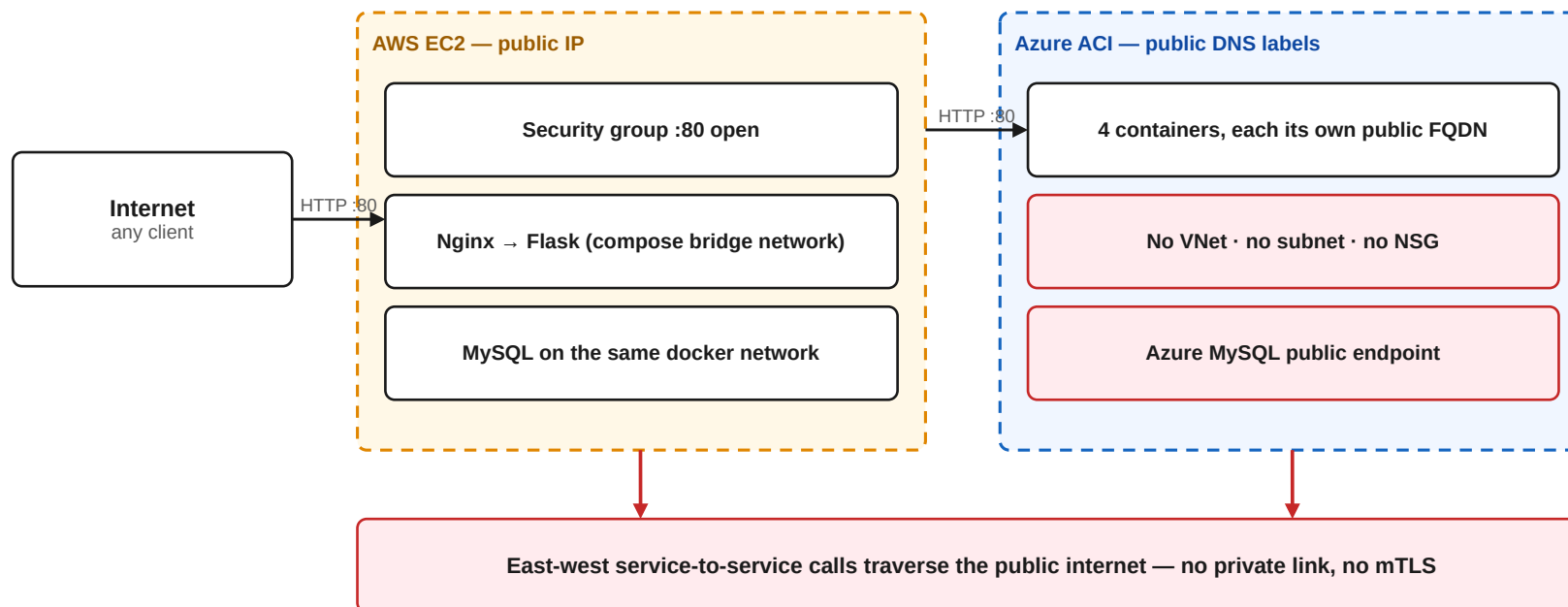
MySQL 5.7 end-of-life is the single most schedule-sensitive item in this table.

Network and Remote Access

Current network posture

Description/s

: *There is no private networking anywhere in the current design. Every component is reachable over the public internet on cleartext HTTP, and east-west service traffic leaves the trust boundary and returns over the same public path.*



Absent controls

- TLS / HTTPS anywhere in the path
- WAF and DDoS protection
- API gateway or managed ingress
- VNet, subnets, NSGs, private endpoints
- Rate limiting at the edge
- Bastion or just-in-time admin access
- Egress control and IP allow-listing

Administrative access to Dev is by SSH to a hardcoded public IP with host-key checking disabled.

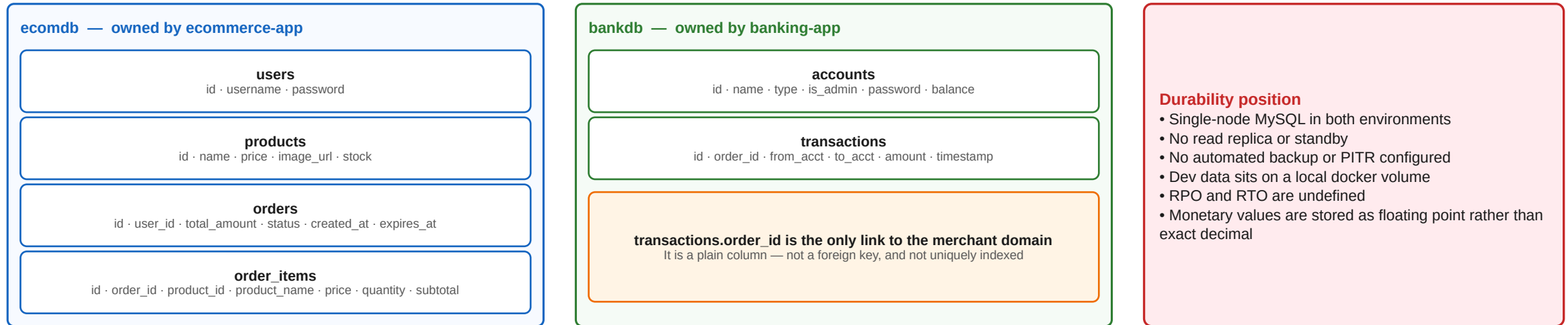
Target direction

Front Door or Application Gateway with WAF terminates TLS at the edge; workloads move into a VNet with private endpoints to the data tier; east-west traffic stays inside the VNet and is mutually authenticated. This is the prerequisite for every other hardening item.

Data and Databases

Data ownership, schema and durability position

Description/s : *Each service owns a private schema, and there is no shared database and no cross-service foreign key. The order lifecycle and the ledger are therefore two independent consistency domains joined only by an HTTP call.*



Concern	Current position	Required for production
High availability	None — single instance per environment	Zone-redundant managed instance with automatic failover
Backup and recovery	Not configured; no restore has been tested	Automated backups, point-in-time restore, periodic restore drills
Network exposure	Public endpoint / shared docker network	Private endpoint, no public network access
Cross-domain consistency	Two commits, no coordination	Outbox plus broker, idempotent consumers, reconciliation sweep

Reporting and Processes

Observability and operational reporting position

Description/s : *Diagnostics today consist of unstructured print statements to container stdout. There is no metrics pipeline, no distributed tracing, no log aggregation and no alerting.*

Capability	Current state	Operational consequence	Recommended platform service
Application logging	print() to stdout, unstructured, lost on replace	No post-incident reconstruction is possible	Container Insights → Log Analytics
Log retention	None — no aggregation or retention policy	Nothing survives a redeploy, i.e. every release	Log Analytics workspace with retention
Metrics	None collected	Saturation, latency and error rate are invisible	Azure Monitor with workload dashboards
Distributed tracing	No correlation ID crosses the two services	A payment cannot be followed across services	OpenTelemetry → Application Insights
Health probing	/health exists; probes only in unused manifests	Unhealthy containers are never replaced	Probes enforced by the orchestrator
Alerting and on-call	None	Failures are found by users, not the platform	Alert rules bound to SLOs, routed to on-call
Availability reporting	No SLI, SLO or error budget defined	Reliability cannot be stated or improved	SLO dashboards with error-budget alerts
Audit trail	transactions table only; mutable	Cannot answer a point-in-time audit question	Append-only store, immutable retention

Position

The platform currently has no means of detecting that a payment failed, and no means of reconstructing one after the fact. Observability is the prerequisite for operating this service at all, ahead of any feature work.

Fault Management

Failure modes and blast radius

Failure event	Platform response today	Blast radius	Detection
Nginx container stops	None — no supervisor beyond docker restart policy, no second instance	Full outage of that service	User report
Application container stops	ACI restarts the instance; clients pinned to that DNS label fail meanwhile	Full outage for that label	User report
Application becomes unhealthy but stays up	Nothing — no readiness probe is executed in the deployed path	Silent black-holing of requests	None
Database instance fails	No standby, no failover; connections simply fail	Full outage plus potential data loss	User report
Database volume is lost	No backup and no tested restore exists	Permanent ledger loss	None until reconciliation
DB_HOST fails to resolve at start-up	Silently falls back to a pod-local SQLite file and reports healthy	Divergent, ephemeral ledger per replica	None — /health returns ok
Merchant callback times out after committing	Bank aborts and tells the customer the payment failed	Permanent cross-database disagreement	None
Production release	Container is deleted then recreated	Planned outage on every deploy	Expected
Availability-zone loss	Not addressed; no zonal or regional redundancy	Total outage	Cloud provider status

Common root cause

Every row above traces to the same two absences: no redundant, load-balanced instance at any tier, and no platform-level health enforcement.

Highest-leverage remediation

A managed orchestrator with health-gated rolling updates plus a zone-redundant managed database closes seven of the nine rows without any application change.

Performance and Reliability

Current characteristics and known load behaviour

Description/s : *No load test, latency budget or throughput target exists. The figures below are structural characteristics derived from the deployed configuration, not measurements.*

Dimension	Current position	Assessment
Service level objective	None defined; no SLI, SLO or error budget	Reliability cannot be stated or measured
Capacity per instance	1 vCPU / 1 GiB per ACI container, Gunicorn default worker count	Unsize — worker count is not tuned to the CPU allocation
Horizontal scale	Fixed at two containers per service; no autoscaling and no load balancer to use them	Effective concurrency is one instance
Latency profile of a payment	Up to three sequential cross-service HTTP calls with 2 s + 3 s + 5 s timeouts	Worst-case tail latency of roughly 10 s before any database work
Polling overhead	Each open checkout page issues 0.5 requests per second, each hitting the database	Load scales with idle browser tabs rather than with transactions
Database connections	Default SQLAlchemy pool per container; no proxy or external pooler	Connection storms likely as instance count grows
Caching	None at any layer	Every read reaches the database
Static asset delivery	Served by the application container; no CDN	Avoidable load on the application tier
Backpressure	No rate limiting, queueing or shedding anywhere	A traffic spike degrades into failure rather than delay

Note

The 10-second worst-case payment latency is a design consequence of chaining synchronous calls, not a tuning problem; it resolves only by making the settlement path asynchronous.



Scalability, Resilience and Disaster Recovery

Description/s : *The platform is single-region and single-instance in effect. No disaster recovery plan, runbook or recovery objective currently exists.*

Scalability

- Fixed instance count, set imperatively in the pipeline
- No autoscaling on any signal
- No load balancer, so the second instance receives no traffic
- Session state in an app-local signed cookie prevents free horizontal scaling
- Data tier cannot scale reads — no replica

Resilience

- Single failure domain per environment
- No zone redundancy at app or data tier
- Health probes defined but never executed
- No retry, circuit breaker or bulkhead on cross-service calls
- Deployment model removes capacity before restoring it

Disaster recovery

- No secondary region
- No database backup configured
- No restore has ever been tested
- RPO and RTO undefined and unagreed
- No documented recovery runbook

Objective	Current	Interim target	Production target
Availability	Not measured	99.0 % single region, health-gated rolling deploys	99.9 % zone-redundant, no planned downtime
RPO	Undefined — data loss is unbounded	24 hours via automated daily backup	5 minutes via point-in-time restore
RTO	Undefined — no restore procedure exists	4 hours with a documented, rehearsed runbook	1 hour with automated failover
Deployment impact	Full outage per release	Zero-downtime rolling update	Blue-green or canary with automated rollback

Interim targets are deliberately modest — they are what the existing Kubernetes manifests could deliver once actually deployed.

Architecture Maturity Assessment

Current state against a production operating bar

Domain	Level	Current position	Production bar
Compute and orchestration	1 / 5	ACI with fixed instances; no ingress, autoscale or rolling update	Managed orchestrator, health-gated rolling deploys
Networking	1 / 5	Public cleartext HTTP end to end; no VNet or private endpoints	Private VNet, TLS at a WAF-protected edge
Data platform	1 / 5	Single-node MySQL 5.7 (end of life); no HA, backup or PITR	Zone-redundant managed database with tested restore
Service integration	1 / 5	Synchronous, bidirectional HTTP; no broker, retry or reconciliation	Asynchronous events with outbox and idempotency
Delivery and supply chain	2 / 5	Gated Dev → Prod promotion, but destructive deploys from a mutable tag	Immutable signed digests, scanned images, progressive delivery
Configuration and secrets	1 / 5	Credentials inline in pipeline definitions; no secret store	Managed identity plus a secret store, no static credentials
Infrastructure as code	1 / 5	None — imperative CLI calls create cloud resources	Declarative IaC in version control with drift detection
Observability	1 / 5	Unstructured stdout only; no metrics, tracing or alerting	Full telemetry pipeline with SLO-based alerting
Resilience and DR	1 / 5	Single region, single instance; no backup, RPO/RTO undefined	Multi-zone with agreed and rehearsed recovery objectives
Health and self-healing	2 / 5	/health endpoints exist and probes are written, but are not deployed	Probes enforced by the orchestrator with automatic replacement

What is already right

Clean service separation with private schemas, health endpoints, environment-driven configuration, an idempotent seed, and a Dev → Prod pipeline with a validation gate. The foundations are sound.

What blocks production

No redundancy, no private networking, no backup, no observability, and an unsupported database version. These are platform gaps, not application defects.

Recommended Direction

Sequenced by risk reduction per unit of effort

PHASE 1

Stop the bleeding

Weeks 1–2 · configuration only

- Enable automated database backups and perform a test restore
- Terminate TLS at a managed edge; disable plain HTTP
- Move credentials out of pipeline definitions into a secret store
- Remove the silent SQLite fallback so failures are loud
- Ship container logs to a Log Analytics workspace

PHASE 2

Make it operable

Weeks 3–8 · platform work

- Deploy the existing manifests to a managed orchestrator
- Health-gated rolling updates to end deployment downtime
- Managed database with zone redundancy and private endpoint
- Workloads into a VNet; private east-west traffic only
- Private registry with immutable digests and image scanning
- Metrics, tracing and correlation IDs across both services

PHASE 3

Make it correct at scale

Quarter 2 · architecture change

- Replace the synchronous callback with broker-based events
- Transactional outbox plus idempotent consumers
- Scheduled reconciliation between the two domains
- Autoscaling driven by real throughput signals
- Declarative IaC for every environment
- Agreed RPO/RTO with a rehearsed recovery runbook

Sequencing rationale

Phase 1 is configuration only and removes the risk of unrecoverable data loss and cleartext credentials within a fortnight — no application change is required.

Phase 2 is platform work that reuses the Kubernetes manifests already written, and it closes seven of the nine fault-management rows.

Phase 3 is the only phase requiring application change, and it is the only way to make the two-database settlement path correct under failure.

Other Design Details

- Testing Approach** :
- *No automated test suite, and no test stage in the pipeline*
 - *The only gate is a curl reachability check against the Dev endpoint after deployment*
 - *For this review both services were executed end to end and the payment journey observed and captured*
 - *Required before production: integration tests across the service boundary, a failure-injection exercise on the settlement path, and a load test to establish a latency budget*
- Deployment Approach** :
- *Jenkins builds one image per service and promotes the same artefact from Dev to Prod*
 - *Dev deployment is SSH plus Docker Compose onto a single EC2 host*
 - *Production deployment deletes and recreates each Azure Container Instance, which incurs downtime on every release*
 - *No rollback path — the :latest tag is mutable, so the previous artefact is not addressable*
- Cost Position** :
- Current footprint is small: one EC2 instance, four 1 vCPU container instances and one managed MySQL instance. The Phase 1 and Phase 2 recommendations are largely configuration and managed-service adoption rather than net new capacity, so the material cost increase is the managed database HA tier and the log analytics workspace.*
- Decisions Requested** :
- ***Confirm whether the Azure and AWS subscriptions in the pipeline are lab or production***
 - ***Agree RPO and RTO so the data-tier design can be fixed***
 - ***Endorse the Phase 1 remediation list and nominate the Phase 2 orchestrator target***